LIFTus: An Adaptive Multi-aspect Column Representation Learning for Table Union Search

Ermu Qiu[†], Jun Gao[†*], Yaofeng Tu[‡], and Jingru Yang[§*]
[†]Key Laboratory of High Confidence Software Technologies, CS, Peking University, China [‡]ZTE Corporation,
[§]National Key Laboratory of Data Space Technology and System
qem@stu.pku.edu.cn, gaojun@pku.edu.cn, tu.yaofeng@zte.com.cn, okiyang@pku.edu.cn

2025 IEEE 41st International Conference on Data Engineering (ICDE) | 979-8-3315-3603-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICDE65448.2025.00165

***Abstract*—Table union search (TUS) represents a fundamental operation in data lakes to find tables unionable to the given one. Recent approaches to TUS mainly learn column representations for searching by introducing Pre-trained Language Models (PLMs), especially on columns with linguistic data. However, a significant amount of non-linguistic data, notably represented by domain-specific strings and numerical data in the data lake, are still under-explored in the existing methods. To address this issue, we propose LIFTus, an adaptive multi-aspect column representation for table unionable search, where aspect refers to a concept more flexible than data types, so that a single column can exhibit multiple aspects simultaneously. LIFTus aims at combining different aspects of a column (including both linguistic and non-linguistic aspects) to promote the effectiveness and generalization of TUS in a self-supervised manner. Specifically, besides employing PLMs to extract the linguistic aspects from an individual column, LIFTus trains a pattern encoder to learn possible character-level sequential patterns for the column, and builds a number encoder to capture numerical aspects of the column, including the distribution and magnitude features. LIFTus further utilizes a hierarchical cross-attention aided by aspect-relevant statistics to combine these aspects adaptively in producing the final column representations, which are indexed by vector retrieval techniques to achieve efficient search. Extensive experimental results demonstrate that LIFTus has outperformed the current state-of-the-art methods in terms of effectiveness, and achieved much better generalization capability to support unseen data.**

***Index Terms*—Table Union Search, Data Lake, Aspect, Non-Linguistic Column, Generalization**

I. Introduction

With the rapid growth of data from various sources, data lake [1]–[3] becomes an attractive infrastructure to accommodate flexible types of data in their raw form directly. Unlike data warehouses [4], data lakes allow storing data including relational tables (sometimes with metadata unavailable) and other types without transforming into a predefined schema [5]. Thus, a fundamental step for an analytic operation in data lake is to locate the relevant data. TUS is such an operation to identify a set of unionable tables in a data lake that share similar columns with the given query Table [6].

TUS methods attempt to achieve effectiveness, efficiency and generalization. Data instances collected in tables within a data lake often exhibit considerable heterogeneity, encompassing linguistic data (*e.g.* words, long texts, dates), non-linguistic data (*e.g.* domain-specific strings, numerical values), and even their combinations [2], [7], [8]. It is challenging to find the desired unionable target tables effectively. In addition, the column representation learning, a key step in the TUS, needs to capture features of flexible data instances and utilize external knowledge, indicating high time consumption. Moreover, TUS methods should be adaptive to new and evolving data, as the volume of data in the lake grows rapidly. Consequently, a generalized model for column representation is highly desirable.

In the context of the TUS problem, unionability refers to the potential for attributes or tables to be unioned based on shared domain features [9]. Early approaches to TUS have focused on designing rules to measure the unionability between two tables [2], [9], [10], but face challenges in capturing latent linguistic information. With the development of PLMs such as BERT [11] and its variants [12], [13], recent works [14]–[17] have employed these models into TUS, as the data in the lake have flexible format including the textual features [18]. Besides PLMs, knowledge bases (KBs) [19], [20] can also provide information for the determination of unionability [7]. Some studies further notice the roles of the inter-column relationships and compose a token sequence by sampling data from different columns into the BERT-like models, thereby learning the context-aware column representations [14], [15]. The learned column representations are then fed into the vector index for fast similarity search [14]. Some studies calculate pairwise similarities of columns to build a semantic graph for TUS [19], achieving promising performance but at the cost of additional preprocessing and online search time [7].

While PLM-based methods become mainstream in TUS, their preference for linguistic data imposes potential limitations on the performances, especially in the context of the relational tables. We illustrate these issues in Fig. 1. Table A and Table B can be unionable as they both pertain to the entities of washing machines manufactured by the same company and their corresponding columns are unionable, while Table C serves as an example that is prone to model misjudgment. We can see that three tables possess linguistic data in their *Location* and *Description* columns, and Table C also includes the *Color* column as part of the linguistic data. Besides, Fig. 1 illustrates the widespread presence of non-linguistic data in the data lake, including but not limited to identifiers (*e.g. Product Code* column) and numerical data (*e.g. ID* column, *Capacity* column, *Price* column, etc.). For

* Corresponding authors

2375-026X/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDE65448.2025.00165

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

example, the strings from the *Product Code* and *Function Abbr* columns (*e.g.* "SDL/JHX/BP/ZH" and "MB65V35E") are not tokens in natural language but highly domain-specific, as they are codes designed by washing machine manufacturers.

Table A

ID	Product Code	Capacity(L)	Function Abbr	Location	Price	Description
20001	EMS75B37mate6	7.5	SDL/JHX/BP/ZH	WalMart	3100	This is a top-loading washing machine that deli...
20002	ES100B35Mate5	10	DDL/JHX/DP	WalMart	2950	Featuring a 10L capacity, it can handle large loa...
20003	ES90B26Mate6	9	SDL/BP/FCR	WalMart	2200	The machine's smart sensors adjust the water...

Table B

ID	Product Code	Capacity(L)	Function Abbr	Location	Price	Description
20981	EMS65B26Mate6	6.5	DDL/DP/DLCJ	Carrefour	1850	With a variety of wash cycles, including a quick...
20982	EB80B37Mate5	8	DDL/DP	Carrefour	2100	Its durable stainless steel drum is gentle on fab...
20983	ES95B36PLUS5	9.5	SDL/BP/FCR/ZH	Carrefour	3500	Equip with a self-cleaning function, eliminating...

Table C

ID	Product Code	Power(W)	Color	Location	Price	Description
301	MB65V35E	340	White	WalMart	1900	The machine's sleek design and compact size m...
302	MB75V35E	400	Silver	WalMart	2150	For those with allergies, the UltraClean 3000's...
303	MB65V55E	360	Black	WalMart	2350	With a quiet operation and a 5-year warranty, t...

Fig. 1. An example of data lake. In this data lake, Table A and Table B are unionable with each other, while Table C is easily misjudged as being unionable with Table A. This misjudgment often stems from an inadequate understanding of non-linguistic columns (*e.g. ID*, *Product Code*, *Function Abbr*), which are widely distributed across the data lake and play a crucial role in determining the unionability of tables. The metadata present here is for easy understanding, but not used in the following processing.

We illustrate the issues faced by the existing PLM-based methods. First, the domain-specific strings, represented by commonly used identifiers, fall outside the typical token set in the PLMs [11]–[13], and cannot be captured by PLMs effectively. Taking Fig. 1 as an example, as the token sets in the PLMs do not cover the values in *Product Code* column and *Function Abbr*, PLMs handle them using the frequent sub-strings, like BPE [21], computed from the large common corpus, which may be different from the key sub-parts in these domain-specific strings. We can see that the similarity between "EMS100B37mate6" and "EMS65B26Mate6" should be significantly greater than that between "EMS100B37mate6" and "MB65V35E". However, PLMs may decompose the string into different parts (*e.g.* "E", "MS1"), which damages the understanding of the domain-specific string. It is imperative to employ character-level sequential modeling for domain-specific strings to capture their similarities.

Second, the accurate perception of the numerical values to match columns is an open problem [22]. It is a live issue for PLMs to adequately understand numerical values [23]. PLMs' token sets can not cover infinite numerical values distributed in continuous space. Thus, the PLM-based methods [14]–[16] struggle to fully understand real numbers, which leads to a loss in the similarity computation over arbitrary numbers in real-life applications. The set-coverage-based methods [9] may yield Jaccard similarity scores nearly zero (*e.g.* {7.5, 10, 9} in *Capacity* column in Table A, {6.5, 8, 9.5} in *Capacity* column in Table B in Fig. 1), despite the fact that these numbers differ only slightly. We also notice that there has been research on encoding numerical types in tables [24]. However, there remains a significant gap between encoding individual numbers and encoding entire numerical columns in TUS, where the latter needs to capture the crucial criteria to determine the similarity among columns, including distribution and magnitude features.

Third, columns may include combined features, and relying solely on data-type-specific approaches may be too coarse to accurately capture column features. As we can see from Fig. 1, some columns in a data lake are rich in semantic information, characterized by a high proportion of natural language words, such as the *Description* columns, which rely more on external corpora or PLMs. In contrast, the *Function Abbr* columns in Table A and Table B should guide the model to prioritize character-level similarity during unionable search. Additionally, some columns are predominantly numerical, such as the *ID* and *Price* columns across the three tables. In such a case, the determination should focus on the distribution and magnitude features of the numbers. Moreover, certain columns contain a mixture of numerical data and non-language strings, such as the *Product Code* columns in the three tables. We can see that even if we can obtain the data type for the column, the data-type-specific methods are not sufficient to capture the flexible features inside the column.

To overcome the aforementioned challenges and issues of the existing methods, we make the following design choices. (1) We introduce a notation of *column aspect*, which is finer than column data type, as each column has only one data type but with multiple aspects. Thus, instead of invoking one method after determining the column data type, we apply multiple aspect extractors on one column simultaneously, so as to handle the different forms of the same typed column or mixed-typed data. We also expect that the full exploration of aspects in an individual column can enhance the model's generalization to unseen data. (2) We do not solely rely on PLM to handle all kinds of columns, but introduce effective methods to capture distinct features in columns, including the linguistic data, domain-specific strings, and numerical values. For aggregating these aspects, we design a learning method rather than heuristic rules to enable robust adaptation to scenarios involving tables with diverse information. (3) We mainly study the exploration of features in one column, and leave the inter-column relationships into future work, since different features inside one single column, we believe, are still underexplored. (4) The method will yield the column embedding, instead of the pairwise similarity model in order to leverage the existing vector index like HNSW (Hierarchical Navigable Small World) [25] to improve the efficiency of online search.

This paper proposes **LIFTus**, an adaptive mu**l**ti-aspect column representat**i**on **f**or **t**able **u**nionable **s**earch. The contributions are as follows:

- We design methods to capture the key non-linguistic aspects in LIFTus, including a pattern encoder for domain-specific strings to infer similarities between strings, and a number encoder to convert the numerical aspect into embedding that preserves the distribution and magnitude features.
- We devise a hierarchical cross-attention integrator, to adaptively combine different aspects of columns, in which

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

the statistics indicating the weights of different aspects are extracted and fed into a hierarchical cross-attention framework to produce the column representations.
- We conduct experiments on four real-world datasets, and the results indicate that LIFTus has achieved a 3.2% improvement in effectiveness on average (**4.7% improvement** for 2 large datasets on average) over the second to the best model at a similar offline preprocessing time cost. The results also show much better generalization capability of LIFTus, which is trained on a small dataset but can be applied to a large dataset, thereby saving training time. For example, after training on SANTOS small only, LIFTus can achieve even better performance on SANTOS large than Starmie, which takes **5.6X** more time training on SANTOS large directly. The code is available on https://github.com/MsiQue/LIFTus.

II. Related Work

In this section, we first review the advances of approaches to TUS, then introduce recent progress in a more generalized table representation learning, and finally discuss the methods used to process non-linguistic data within tables.

A. Table Union Search

TUS is a frontier research topic and has witnessed rapid advancements recently [2], [7], [10], [14]. We review the existing methods from various design choices, ranging from the measurement of the unionable table, different forms of external knowledge, the consideration of inter-column relationships, and training strategies.

There are explicit and implicit measurements for the unionable table. Nargesian et al. [9] have introduced the concept of TUS, and proposed three aspects of unionability. Bogatu et al. [10] have introduced D^3L, extending the determination of unionability to a five-dimensional model. Other recent works, including SANTOS [7] and Starmie [14], learn the unionable tables in an implicit way, in which the previous factors are constructed as a column relation graph or fed into PLM.

Different external knowledge can be incorporated into the TUS. The early works [2], [10] mainly utilize external knowledge bases [19], [20] or word embeddings [26]. SANTOS [7] leverages both existing knowledge bases [19] and a synthesized one, generated by exploiting the inherent knowledge within the data lake. Starmie [14] relied on PLMs [11]–[13], which are based on the transformer architecture [27] to learn the attention weights between token pairs.

The inter-column relationships are considered by SANTOS [7] and Starmie [14]. They assume that the neighboring columns provide a context for an individual column. Specifically, SANTOS acquires semantic relationships using the knowledge base, and Starmie vectorizes each column by concatenating tokens sampled from neighboring columns and feeding them into a language model.

The model can be trained in a supervised or self-supervised fashion. D^3L [10] treats the determination of unionable tables as a binary classification problem in a supervised manner. To reduce the high cost of manual labeling, Watchog [28] and Starmie [14] have shifted to the self-supervised fashion, allowing researchers to expand the column encoder for various downstream tasks.

The existing methods bring the following inspiration to LIFTus. LIFTus follows a self-supervised fashion to train a column encoder for TUS with sufficient factors considered. In addition, LIFTus chooses the PLM to handle the linguistic data as a part of producing column representations. LIFTus puts off the inter-column relationship to the future work, as we observe that the column features are not fully explored in the current data preprocessing.

B. Table Representation Learning

TUS is a specific downstream task of the generalized table representation learning, which can support other tasks like column classification, data integration, etc. We can roughly categorize the existing methods into basic neural network based approaches [29]–[32], graph representation learning based approaches [33]–[35], and pretrained model based approaches [22], [36]–[40]. Among them, Sherlock [29] employs a multi-input deep neural network to integrate various feature categories and learns column embeddings in a supervised manner. SATO [30] extends Sherlock [29] by incorporating inter-column information and leverages Latent Dirichlet Allocation (LDA) [41]. EmbDI [33] and HierGAT [34] learn embeddings for both columns and rows by utilizing graph-based random walk and attention mechanism, respectively. Pretrained model based approaches can mainly be divided into two categories. One is to use tables as a corpus to pre-train the models [38]–[40], and the other is to directly utilize PLMs to understand tables for solving downstream tasks [22], [28], [36], [37].

C. Non-linguistic Column Understanding

Domain-specific String. Existing works handle the domain-specific strings by extracting their formatting rules or regular expressions. Cornet [42], [43] leverages user examples to automatically learn formatting rules for a specific column. Bogatu et al. [10] establish a collection of rules to transform columns into a set of regular expressions and measure the similarity of two columns using the Jaccard similarity. Sherlock [29] focuses on the statistics of characters in strings (*e.g.* mean, max, sum, etc.), while the order of characters within a string is not extensively explored in computing similarity between strings. In the context of TUS, we actually do not require explicit regular expressions for strings, and an implicit model is sufficient for the similarity among strings.

Numerical Data. Some works measure the correlation between two numerical columns using statistics. STEER [44] proposes a steered-labeling approach to handle numerical columns effectively with Earth Mover's Distance for similarity assessment, which leverages non-numerical column labels to guide context-aware clustering. Bogatu et al. [10] measure the correlation by computing the Kolmogorov-Smirnov (KS)

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

statistic [45] for numerical columns and treat the result as one of the aspects of unionability. Other works have enabled numerical values to be comprehended by GNNs [46], [47] or LLMs [18], [48]–[50] through encoding them into high-dimensional vectors. Among them, Pythagoras [35] proposes a novel graph representation of tables to combine numerical data with contextual non-numerical features. Yan et al. [24] present TP-BERTa, a PLM that utilizes C4.5 discrete algorithm [51] and relative magnitude tokenization to convert scalar numerical values into tokens. Existing research on modeling numerical data faces two primary challenges. First, capturing the relationship between two numerical columns through a single statistical measure often proves insufficient for precise representation [10]. Second, employing language models to encode entire columns of numerical data needs a significant number of tokens. To address these limitations, we plan to model the distribution and magnitude features of the numerical column respectively.

III. METHODOLOGY

In this section, we first provide an overall framework of LIFTus, and then detail pattern encoder and number encoder to extract features from domain-specific strings and numerical data, respectively. Next, we design an aspect-relevant statistics-aided hierarchical cross-attention method to combine these aspects into column representations. Finally, we provide a complete algorithm of LIFTus to connect the above components and analyze the time complexity.

A. Framework

Fig. 2 illustrates the architecture of LIFTus, where the lower portion depicts the entire process of LIFTus, including the offline data preprocessing phase as well as the online unionable table query phase. The offline data lake preprocessing phase of LIFTus (lower left part of Fig. 2) is to convert the column data into vectorized representations in a self-supervised manner. The online query phase (lower right part of Fig. 2) returns a set of tables that are unionable with the given query table. The upper portion of Fig. 2 highlights the column encoder, a crucial component of LIFTus.

The column encoder extracts and aggregates information from various aspects, thereby adapting to more complex input columns. For example, the input column *User Nickname* in Fig. 2 originates from arbitrarily chosen usernames, which has 4 aspects of information: paragraph aspect (*e.g.* cells "`KeyboardBreaker@304`" and "`Python Pioneer Pro`"), word aspect (*e.g.* tokens "`python`" and "`loop`"), number aspect (*e.g.* tokens "`504`" and "`2601111001`") and pattern aspect (*e.g.* summarized wildcard patterns "`???_??0??CS`" and "`stu_??0??CS_2`"). Therefore, relying on feature extraction from any single aspect is insufficient to accurately capture the information of this column.

For each column, LIFTus extracts both linguistic (paragraph, word) and non-linguistic (number, pattern) aspects. Each aspect corresponds to a dedicated encoder. For the linguistic aspect, LIFTus follows the previous approach by employing BERT [11] as the paragraph encoder and utilizing fastText [26] as the word encoder. In the following part of this section, we focus on the processing of non-linguistic aspect by equipping LIFTus with the pattern encoder (3-B), number encoder (3-C), and hierarchical cross-attention integrator aided by statistic-driven aspect indicator (3-D).

B. Pattern Encoder for the Domain-Specific String

Domain-specific strings serve as the primary source of pattern aspect information within the data lake and thus should be considered in the TUS. As a crucial form of non-linguistic data, domain-specific strings are common across multiple tables within the data lake. Their forms include but are not limited to identifiers, object IDs, and specific abbreviations. Furthermore, columns containing domain-specific strings often serve as crucial criteria to determine the unionable tables. For instance, the *Product Code* column is the candidate key in three tables illustrated in Fig. 1. It is easy to know that these tables are more likely to be unionable if their primary (or candidate) keys share a similar pattern.

The basic idea to capture patterns of domain-specific strings is to train a character-level sequence model over the extracted wildcard patterns. As we discussed before, domain-specific strings are challenging for PLMs to comprehend, as they typically fall outside of open corpora. In addition, we cannot learn the patterns from the raw strings directly, given the vast quantity and flexible representations of strings in the data lake. Moreover, we do not rely on the extracted regular expression to calculate the similarity of two strings [10], which lacks generalization and needs pairwise comparison. By considering the factors above, LIFTus first extracts the pattern with wildcards, and then trains a sequence model over these patterns. The similarity between two columns is reduced to the similarity between the hidden output from the sequence model. The vectorized pattern representation can be flexibly integrated with other aspects, facilitating subsequent use with an efficient index.

The pattern encoder of LIFTus consists of two components: the pattern summarizer and the pattern sequence mode. The former is to extract the wildcard patterns from the sampled values, which identifies the shared patterns of column strings and lowers the size of the data instances for the following sequence model. The latter relies on self-supervised learning with the data augmentation rules to learn a sequence model over the extracted patterns to avoid manual labeling.

Pattern Summarizer. The purpose of the pattern summarizer is to extract the meaningful wildcard patterns from the given strings, inspired by the recent work on the regular expression extraction [10]. Specifically, we sort the column data lexicographically in both forward and backward directions, with the observation that domain-specific strings may share the same prefix or suffix. Subsequently, we collect $n_{Pattern}$ samples from the column data to lower the size of raw data, on each of which the pattern summarizer scans n_{Check} consecutive strings starting from that position to find the wildcard patterns

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

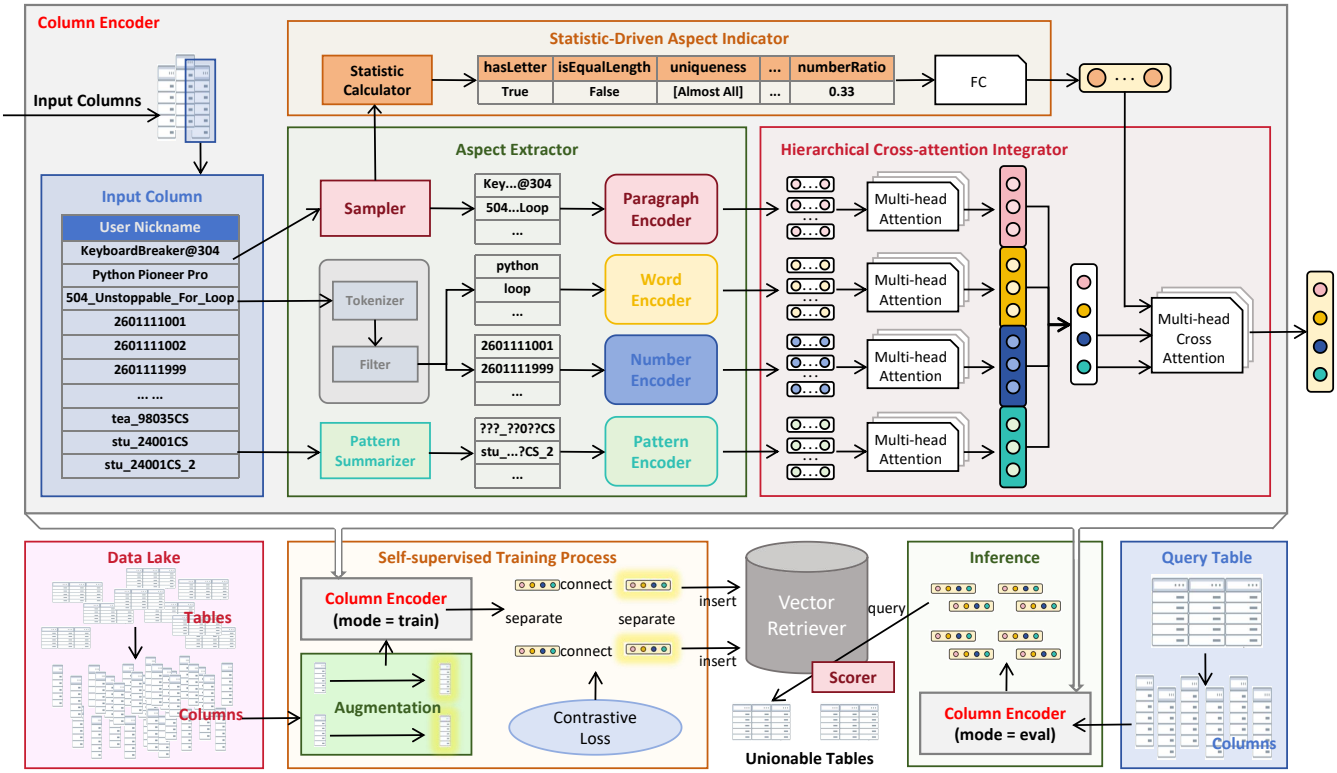


Fig. 2. Architecture of LIFTus. LIFTus has 2 phases: offline data lake preprocessing phase (lower left) and online query phase (lower right). In both phases, column encoder (grey box) serves as a crucial component that incorporates 4 aspects (paragraph, word, number and pattern) into consideration. In the column encoder, the aspect extractor first samples the input column and computes information on 4 aspects. Under the direction of statistic-driven aspect indicator, hierarchical cross-attention integrator aggregates the information in a self-supervised manner to obtain the column embedding.

in two directions. To be more specific, the patterns can be identified through merging operations while comparing strings character-by-character. If the characters at corresponding positions are different, a wildcard is used instead. For example, merging of "`tea_98035CS`" and "`stu_24001CS`" yields "`???_??0??CS`". The pattern extracted will be kept in P for the sequence model learning.

The pattern summarizer runs on all table columns in the data lake, no matter their potential types. When there is no valid wildcard pattern extracted from n_{Check} consecutive strings, we use the string on starting position directly. Such cases can happen when columns processed are with types like long text or numerical data. Directly using the raw data as a wildcard pattern does not negatively impact the results, as the subsequent hierarchical cross-attention integrator dynamically adjusts the weight distribution among the encoders.

Pattern Sequence Model. The pattern sequence model accepts the wildcard pattern discovered by pattern summarizer, and produces the hidden state of the domain-specific string aspect in one column. Besides the explicit characters in the wildcard pattern, we find from the experimental studies that the types (including digits, lowercase/uppercase letters, and other characters) play important roles in the hidden representation generation. Such a feature also follows domain constraints intuitively. That is, the similar identifiers are with the same type of characters. Such a rule-based annotation will give more hints for the following sequence model. Specifically, the pattern encoder processes the wildcard pattern p using the following formula.

$$H(p) = \text{LSTM}(\left[char(p_i) \oplus type(p_i)\right]_{i=0}^{n-1}) \tag{1}$$

Here, n is the length of a wildcard pattern p, $char(p_i)$ represents a one-hot character representation, and $type(p_i)$ represents the type of p_i mentioned above. Subsequently, we concatenate each character in p in order, followed by encoding with a sequential model LSTM [52]. The choice of LSTM relies on its gating mechanism, especially the forget gate, which allows the model to effectively retain key patterns in sequences, such as common substrings.

The pattern sequence model is trained in a self-supervised manner. To achieve this, we first define the data augmentation rules as illustrated in Table I, which enables the model to more effectively extract string features for TUS. Subsequently, we treat the original string and the augmented string as a positive pair, using SimCLR [53] as the loss function for training.

C. Distribution-Magnitude Aware Number Encoder

Similar to domain-specific strings, numbers represent a typical form of non-linguistic data, which are commonly encountered within data lakes. Numerical columns exhibit rich

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DATA AUGMENTATION RULES FOR CHARACTER-LEVEL PATTERN ENCODER

Rule	Augmented string
Original string	tea_9803?CS
Replace a character with wildcard "?"	te?_9803?CS
Convert "?" to a regular character	tea_98035CS
Change a character to another (same type)	sea_9803?CT
Add a character at the end	tea_9803?CTF
Add a character at the beginning	atea_9803?CT
Delete a character	tea_983?CT

diversity with varying distribution and magnitude features. Here, distribution feature describes how values in a column are distributed in an entire set, like a uniform distribution, normal distribution, long-tail distribution, or even distributions that are challenging to be described mathematically. Magnitude feature describes the absolute scale of values in a data column, including metrics such as the mean, maximum, and so on.

Data from similar columns are expected to exhibit similar distributional or magnitudinal features, thereby serving as a potential basis for determining table unionability. For example, for certain columns with continuous values which may be measurements of specific attributes of entities, the normal distribution and magnitude features such as mean, maximum, and minimum play a significant role in assessing the unionability of these columns. For other columns with discrete values like IDs or categorical variables, the uniform or long-tail distribution and magnitude features such as range are also worth considering.

Given the above properties of the number aspect, the number encoder of LIFTus models the distribution and magnitude of numerical columns directly, rather than treating numbers as strings [7], [9], [14] or utilizing a single statistic [45] to measure pairwise correlation between columns [10]. Besides, in order to apply the number encoder to not only numerical columns but also other columns with mixed data types, LIFTus chooses to tokenize sampled cells within the column, and then feeds the resulting tokens into filters to extract numerical values. After that, the values are sent to the following 2 number encoders.

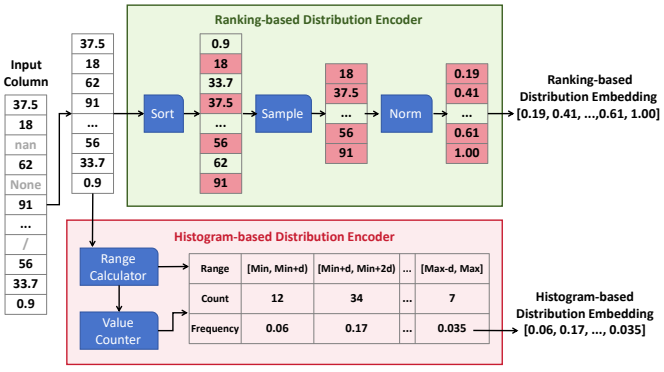


Fig. 3. Number distribution encoder of LIFTus.

Number Distribution Encoder. The number distribution encoder is recursively implemented with ranking-based distribution encoder (ranking encoder) and histogram-based distribution encoder (histogram encoder) in Fig. 3, which extract the distribution from different viewpoints.

Ranking encoder (shown as green box in Fig. 3) models the distribution of the column by selecting representative elements within the column. To achieve a balance between using all available elements and relying solely on the mean as a representative measure, we conduct linear sampling on the sorted elements. More specifically, assume that m denotes the number of numerical elements in the column, and hyperparameter d represents the dimension of target embeddings. The sampling process is conducted using the following formula.

$$sample_i = \text{sorted}(col)_{\lfloor \frac{m}{d} \rfloor \cdot i} \tag{2}$$

Following this, we normalize the above results to obtain ranking-based distribution embedding (RDE).

$$RDE(col) = \left[\frac{sample_i - \min(col)}{\max(col) - \min(col)} \right]_{i=0}^{d-1} \tag{3}$$

Histogram encoder (shown as red box in Fig. 3) serves as a complementary extension to the ranking encoder, as the ranking encoder only reports the data sampled. In order to mitigate the impact of unsampled elements in the ranking encoder, the histogram encoder employs a histogram to compute the frequency of values within equal-width partitions of the column domain, which ensures that each element within a column can contribute to the final embedding.

Specifically, histogram-based distribution embedding (HDE) can be computed using the following formula:

$$HDE(col) = \left[\frac{\text{counter}(col, LB + it, LB + (i+1)t)}{\text{length}(col)} \right]_{i=0}^{d-1} \tag{4}$$

where $\text{length}(col)$ represents the number of numerical values in col, $\text{counter}(col, L, R)$ denotes the count of values within the interval $[L, R)$ in col, $LB = \min(col)$ represents the lower bound of the whole column, the hyperparameter d refers to the desired dimension of the embedding, and $t = \frac{\text{Max}(col) - \text{Min}(col)}{d}$ denotes the interval width.

The computation of the $RDE(col)$ and $HDE(col)$ in Fig. 3 can be efficiently performed by linearly scanning and counting the sorted data.

Number Magnitude Encoder. Since all numerical values are normalized in the number distribution encoder, the absolute values are ignored. Therefore, magnitude features of numerical columns are also key factors in measuring unionability. In the number magnitude encoder of LIFTus, we select the mean, maximum, minimum, and range as representative values to measure the magnitude features of numbers within columns. When two columns have similar magnitude features, their first three statistical measures tend to be similar.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

After the selection of statistical measures for magnitude, the next step is to transform a statistical measure x into an embedding. Some works [24], [54], [55] determine the transformation using learning methods. In the context of TUS, obtaining appropriate pre-trained encodings for numerical values presents a significant challenge due to the limited training data. Thus, LIFTus takes static functions to convert each measurement into d-sized vectors, whose weights are determined by the subsequent hierarchical cross-attention integrator.

Specifically, we use three kinds of functions to calculate value magnitude embedding (ME), each of which has a distinct emphasis. The first function $f^a_{base}(x)$ attempts to produce a d-sized vector, which can reveal the similarity between two columns with similar magnitude features. Here, sgn is a function to produce 1 for the positive input, or -1 for the negative input. The second function $f^a_{macro}(x)$ finds relationships between two columns when the data exhibits a small relative difference but a substantial absolute difference. For instance, the difference between $f^a_{macro}(10^5)$ and $f^a_{macro}(2 \times 10^5)$ is far less than that between $f^a_{macro}(10^5)$ and $f^a_{macro}(0)$. Unlike the second function which focuses on modeling the scale of numerical values, the third function $f^a_{micro}(x)$ is highly sensitive to minor variations in the numbers. Only two numbers with an extremely small absolute difference will have similar values of the $f^a_{micro}(x)$ function.

$$f^a_{base}(x) = \mathrm{sgn}(x)|x|^{\frac{a}{d}} \tag{5}$$

$$f^a_{macro}(x) = \mathrm{sgn}(x)\log_a(|x|+1) \tag{6}$$

$$f^a_{micro}(x) = \sin(ax) \tag{7}$$

In order to convert a statistical measure x into a set of d-sized vectors, we enumerate a from 1 to d, calculate the corresponding values of three functions, and concatenate them to obtain three vectors.

$$ME_{op}(x) = [f^a_{op}(x)]^d_{a=1}, op \in \{base, macro, micro\} \tag{8}$$

The concatenated vectors exhibit different emphases. ME_{macro} emphasizes amplifying the differences in values on a macroscopic level, mapping columns with similar value ranges (*e.g.* both columns with values on the order of millions) to close high-dimensional spaces. Conversely, ME_{micro} focuses on enhancing the differences in values on a microscopic level, like two columns representing sensor measurements that are deemed unionable due to their close values (*e.g.* an average of 9.31 and 9.32, respectively). This design enables the subsequent hierarchical cross-attention integrator to have a more flexible space for weight allocation.

The outputs of the number distribution encoder and the number magnitude encoder are respectively mapped to a high-dimensional space through linear transformations. These transformed outputs are then concatenated to serve as the output of the number encoder.

D. Multi-aspect Integration Mechanism

In a data lake, non-linguistic information exists in various forms, which leads to different methods exhibiting varying effectiveness in determining column unionability. Columns with a higher proportion of strings are suitable for evaluation using the pattern encoder. Columns with a predominance of numerical values are better assessed using the number encoder. Columns rich in semantic content are best evaluated using linguistic information. Additionally, many columns may require a comprehensive evaluation incorporating several of these criteria, like the column *User Nickname* in Fig. 2.

In other words, the criteria dominating the determination of unionability for different columns may originate from different aspects. As we mentioned before, there is a strong correlation between statistics of a column and weights assigned to various aspects of this column. LIFTus decides to use statistics to guide the weight distribution for each aspect.

Statistic-Driven Aspect Indicator (SDAI). We can rely on aspect-aware statistics to determine the predominant aspect in the unionable search process. For example, we define a statistical metric named *NLW-Coverage*, a ratio to describe to what extent the tokens in the column are natural language words. The *NLW-Coverage* can be a good hint to determine the domain-specific strings or semantic-rich sentences. We will get a low *NLW-Coverage* ratio on *Product Code* or *Function Abbr* column, but obtain a high *NLW-Coverage* ratio on *Description* column in Table A and Table B in Fig. 1.

We illustrate the role of another two metrics, *numberRatio* and *uniqueness*, where the former reveals the chances that the characters in the tokens belong to digits, and the latter indicates the extent that values are distinct. The *ID* columns of the three tables in Fig. 1 will have their *numberRatio* 1 and large *uniqueness*, which consequently gives hints on high weights of pattern encoder and number encoder.

In all, we compute a total of 23 metrics and illustrate partial metrics in Table II, which all can be computed in linear or logarithmic time. We categorize the statistical metrics into two types: discrete statistics and continuous statistics. Discrete statistics have a finite number of values, including binary variables with values of True/False, as well as certain manually defined metrics. For example, we have defined multiple levels for *uniqueness*, including [All same], [2-Valued], [3-Valued], ..., [Almost all diff], [All diff]. Discrete statistics are represented using the one-hot encoding method. The range of continuous statistics encompasses the entire real number space, so we directly use its computed value as a feature.

TABLE II
PARTIAL STATISTICAL METRICS IN THE SDAI

discrete statistics	uniqueness, isEqualLength, hasLetter, hasSpace, hasDigit, hasSpecialCharacter, ...
continuous statistics	averageLength, varianceLength, numberRatio, floatRatio, stringRatio, NLW-Coverage, ...

The SDAI is presented in the yellow box located at the top of Fig. 2. SDAI initially employs the statistic calculator

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

module to compute the relevant statistical metrics for the input column. After, the statistical metrics are concatenated into a vector X_{SDAI} and then mapped to a d-dimensional space through fully connected layers W_0, obtaining F_0 as a part of the subsequent cross-attention. The formula is as follows:

$$F_0 = W_0 X_{SDAI} \tag{9}$$

Hierarchical Cross-attention Integrator (HCAI). As previously concluded, information from SDAI has the potential to guide the weight distribution for each aspect in unionability determination. We observe that information from SDAI and the other 4 aspects all originate from the column itself, whose relationships are similar to different modal features in the multi-modality tasks. Thus, we borrow the idea of multi-modality cross-attention [56], [57], and attempt to learn the weight of different aspects using the similarities between the SDAI vector and the aspect vectors after mapping into a high-dimensional space.

Based on this consideration, we propose the HCAI, which aggregates the outputs from the preceding encoders. We denote the output results of the aforementioned four encoders as X_T, which is a matrix with the shape of $n_T \times d_T$, where $T \in \{Paragraph, Word, Number, Pattern\}$, d_T refers to the embedding dimension of each aspect, and n_T refers the number of sampled units. A sampled unit refers to the smallest element during the sampling process within a specific aspect. (*e.g.* cell for paragraph encoder, token for word encoder, summarized wildcard pattern for pattern encoder, and the embedding of distribution or magnitude of statistics for number encoder).

The HCAI of LIFTus's column encoder has two layers. The first layer focuses on the relationships among sampled units for an individual aspect. As we assume that the influence between sampled units from different aspects is negligible (*e.g.* influence between a histogram distribution embedding of all numbers in a column and a wildcard pattern sampled from that column). To reduce computational complexity, we do not integrate information from different aspects in this layer. We use the self-attention mechanism to integrate the unified embeddings of aspects using the following formula:

$$F_T = \text{Self-Attention}(X_T) \tag{10}$$

The second layer of HCAI derives the ultimate vectorized representations of the input columns. Specifically, we map the embeddings from the previous layer to the same d-dimensional vector space using learnable parameter matrices and then concatenate them. The formula is as follows:

$$F_{cat} = [W_1 F_{Paragraph} || W_2 F_{Word} || W_3 F_{Number} || W_4 F_{Pattern}] \tag{11}$$

As we mentioned before, weight distribution for unionability assessment of different aspects varies across different columns, influenced by the statistical information of the respective columns. Instead of manually designing the function between statistical information and weight distribution, HCAI applies cross-attention between the column statistics embeddings (F_0) and the unified embeddings (F_{cat}) to produce the final column embeddings using the following formula:

$$F = \text{softmax}(\frac{F_0 F_{cat}^T}{\sqrt{d}}) F_{cat} \tag{12}$$

Note that the aforementioned matrices F_0 and F_{cat} are of dimensions $1 \times d$ and $n \times d$ respectively, where d donates the dimension of the hidden layer and n refers to the sum of the number of sampling units in each aspect ($n = \sum n_T$). Therefore, the resulting matrix F can be directly squeezed into a one-dimensional vector of size $1 \times d$ for subsequent vector retrieval algorithms (*e.g.* HNSW [25]). In section IV, we further explain HCAI through an example.

E. Implementation and Complexity

This subsection provides a comprehensive overview of the implementation process of LIFTus, thereby connecting the individual components introduced earlier. LIFTus handles the table union search problem through two phases: pre-processing and query.

Offline Pre-Processing Phase. The Pre-Processing Phase consists of 3 stages: (1) splitting the data lake into multiple columns, (2) performing contrastive learning on these columns to conduct self-supervised training of the multi-aspect column encoder, and (3) generating column embeddings using the column encoder and constructing an index with the HNSW algorithm [25]. In the following, we will detail the second step.

In the training process, $C_{original}$ and $C_{augment}$ are obtained by augmenting a batch of columns C. Here, we choose random row sampling as the $augment_operator$. After that, $C_{original}$ and $C_{augment}$ are sent to AspectExtractor, which is a critical component transforming a column C into representation across various aspects X_{SDAI}, $X_{Paragraph}$, X_{Word}, X_{Number} and $X_{Pattern}$. The procedures are as follows. Firstly, based on the ideas in section 3-D, we choose n_{SDAI} cells from a column to compute its statistical features. Then, $n_{Paragraph}$ cells are sampled from the column, and their embeddings are directly obtained using a language model [11]. Subsequently, we tokenize the n_{WN} sampled cells and then filter them to obtain n_{Word} words and n_{Number} numbers, which are respectively fed into the corresponding encoders. Finally, the pattern summarizer and pattern encoder capture the pattern aspect of the column, as elaborated in section 3-B. AspectExtractor is controlled by the hyperparameter set hps, including n_{SDAI}, n_{WN}, $n_{Paragraph}$, n_{Word}, n_{Number}, $n_{Pattern}$ and n_{Check}.

Through AspectExtractor, $C_{original}$ and $C_{augment}$ can be transformed into $X_{original}$ and $X_{augment}$. Subsequently, $V_{original}$ and $V_{augment}$ are obtained using the HCAI in section 3-D (Equations 9 to 12). The loss L is then computed using SimCLR [53], and its backpropagation is employed to update the parameters in column encoder $\mathcal{E}$.

The time complexity of the offline preprocessing phase is the total of its three stages. Let $\mathcal{N}$ denote the total number of columns in the data lake. Stage 1 involves splitting all tables in the data lake, which takes $O(\mathcal{N})$ time. In stage 2,

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

AspectExtractor ensures that all aspects are computed using a fixed number of sampled units. hps are constants, independent of the total number of rows in a table, and remain fixed across all datasets. Additionally, during self-supervised contrastive learning, the number of epochs, batch size, hidden layer size, and embedding dimensions are all fixed. Since these quantities do not grow with the size of the data, the time complexity of stage 2 can be represented as $O(c\mathcal{N})$. Stage 3 uses the HNSW algorithm [25] to build an index, with a time complexity of $O(\mathcal{N}\log(\mathcal{N}))$. Thus, the overall time complexity of the Pre-Processing Phase is $O(\mathcal{N}(c+\log(\mathcal{N})))$.

Online Query Phase. To ensure a fair comparison, this phase is implemented using a method similar to Starmie [14]. After the pre-processing phase, LIFTus can efficiently search online for tables that are unionable with the given query table. Specifically, LIFTus first splits the query table into several independent columns, and then uses the HNSW algorithm [25] to find the top-k columns closest to each query column. Finally, the scores of the tables associated with these columns are aggregated to obtain a unionability prediction value. With the optimization provided by the HNSW algorithm [25], the time complexity for nearest neighbor search on a table with m column is reduced to $O(m\log(\mathcal{N}))$, where $m \ll \mathcal{N}$.

IV. Experiment

In this section, we first introduce our experimental setups. We then assess the effectiveness and efficiency of LIFTus, and study the generalization capability of different baselines. Finally, we perform the ablation studies to show the effectiveness of different components of LIFTus.

A. Experimental Setups

Datasets. As shown in Table III, we use 4 datasets to evaluate LIFTus. All of them are widely used in the TUS task in previous works [2], [7], [14]–[16].

TABLE III
Overview of datasets

Benchmark	#Tables	#Columns	#Query Tables
TUS small (t)	1,530	14,810	150
TUS large (T)	5,043	54,923	170
SANTOS small (s)	550	6,322	50
SANTOS large (S)	11,090	123,477	36

Through projection and selection operations, TUS small and TUS large were derived from 10 and 32 tables, respectively. SANTOS small dataset is an extension of 296 real data lake tables from 35 different domains [7]. Ground truth for the unionable tables in above datasets is publicly available. Due to the lack of public ground truth results for SANTOS large, we randomly select 36 tables as query tables and annotated 20 tables unionable with each query table as ground truth.

Baselines. To evaluate the effectiveness of LIFTus, we compare it with D^{3}L [10], Sherlock [29], SATO [30], SANTOS [7] and Starmie [14]. To ensure a fair comparison, we adopt the same evaluation setting used by previous studies [14], [29], [30] capable of generating column embeddings.

In order to evaluate the effectiveness of each component of LIFTus, we design the following variants of LIFTus. "w.o. PE" refers to the variant of LIFTus with the pattern encoder removed. Similarly, NE, H, and S are abbreviations for "number encoder", "HCAI", and "SDAI", respectively.

Metrics. We use precision at k ($P@k$), recall at k ($R@k$) and mean average precision ($MAP@k$) to measure the effectiveness of top-k unionable tables returned, as other previous works [2], [7], [14]–[16]. Specifically, for a given query table Q, the model returns a set $\hat{S}$ containing k tables, representing the top-k tables with the highest unionability with Q. We denote the set of truly unionable tables as S derived from the ground truth. $P@k$, $R@k$ and $MAP@k$ can be computed via the following formulas.

$$P@k = \frac{|S \cap \hat{S}|}{k}, R@k = \frac{|S \cap \hat{S}|}{|S|}, MAP@k = \frac{1}{k}\sum_{i=1}^{k} P@i \tag{13}$$

In the same setting as previous work [7], [14], we set k to 60 for the two TUS datasets and to 10 for the SANTOS small dataset. Considering the number of unionable tables to each query, we set k to 20 for the SANTOS large dataset. Note that $R@k$ has a upper bound ($k/|S|$) that is always less than 100%. As with previous work [7], we cancel reporting $R@k$ for SANTOS large dataset as it involves cumbersome manual labeling operations across the entire dataset.

For a fair comparison, all baselines apply HNSW [25] to identify unionable tables. Compared to time-consuming linear scanning, HNSW [25] suits application scenarios better but with certain inaccuracies. The data reported hereinafter are lower than the Table 3 of Starmie [14] in $MAP@k$ and $R@k$, but are consistent with the results in its appendix.

B. Evaluation of Effectiveness and Efficiency

Here, we first compare LIFTus with other baselines in terms of $MAP@k$ and $R@k$, with the fixed k, where k is the number of the searched tables. We then perform studies of all competitors on $P@k$ and $R@k$ across different values of k. Finally, we report the time, memory and scalability of LIFTus.

All competitors run on 4 datasets, except SANTOS on the TUS large due to the absence of the requisite files necessary for execution [7]. In addition, we do not compute the $R@k$ metric on SANTOS large dataset, because it is costly to enumerate over 11,000 tables within the data lake to obtain a complete ground truth set.

Table IV shows that LIFTus delivers state-of-the-art performance in terms of $MAP@k$ across four datasets. We can see that LIFTus achieves an average improvement of 3.2% (1.1%, 5.6%, 2.1%, 3.8%, respectively) over all datasets against the existing previously best-performing baselines. In addition, the advantage of LIFTus is more obvious on larger datasets than that on smaller datasets. For example, LIFTus achieves **5.6% improvement** over SATO in TUS large data,

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

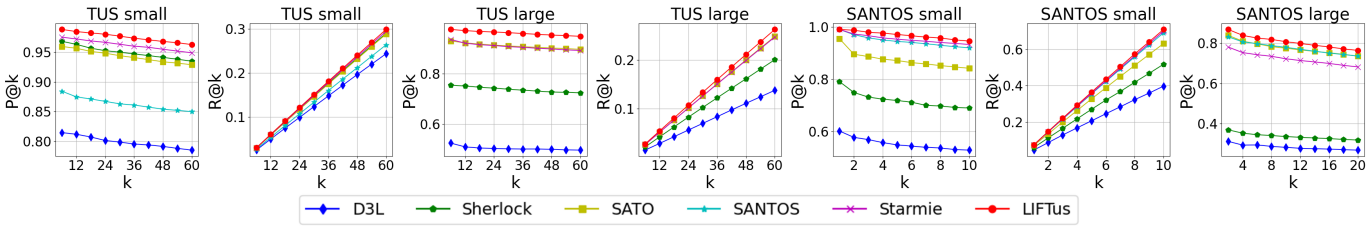


Fig. 4. $P@k$ and $R@k$ of LIFTus against baselines in different datasets.

TABLE IV
PERFORMANCE COMPARISON OF DIFFERENT METHODS. THE DATA WITH BLACK FONT ARE THE BEST RESULTS, AND THE UNDERLINED DATA ARE THE SECOND TO THE BEST

Methods	TUS small		TUS large		SANTOS small		SANTOS large
	$MAP@k$	$R@k$	$MAP@k$	$R@k$	$MAP@k$	$R@k$	$MAP@k$
D^{3}L	0.801	0.243	0.504	0.136	0.556	0.402	0.281
Sherlock	0.951	0.292	0.739	0.203	0.727	0.526	0.334
SATO	0.943	0.287	0.913	0.248	0.877	0.628	0.784
SANTOS	0.867	0.263	-	-	0.951	0.692	0.786
Starmie	0.965	0.294	0.911	0.247	0.948	0.691	0.724
LIFTus	**0.976**	**0.298**	**0.964**	**0.262**	**0.971**	**0.708**	**0.816**

and **3.8% improvement** over SANTOS in SANTOS large dataset.

Table IV also illustrates the results of $R@k$. The values of $R@k$ of LIFTus also surpass these of all baselines, and the advantages are more obvious on TUS large dataset. By combining the results on $MAP@k$ and $R@k$, we believe that LIFTus could capture the key features in the column representations.

Fig. 4 further focuses on $P@k$ and $R@k$ varying k on all datasets, where k is the number of selected tables. Under typical settings in TUS, the $P@k$ curve tends to decrease as k increases while $R@k$ increases with the increase of k. We can see that the red curve, representing LIFTus, consistently occupies the uppermost position across all 7 subfigures. LIFTus demonstrates a notably large margin of improvement in 4 datasets.

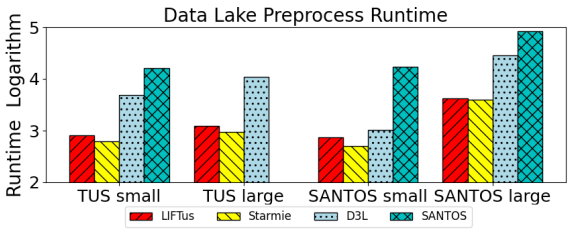


Fig. 5. Data lake preprocess runtime in different datasets.

Fig. 5 reports the running time (logarithm to the base 10) in the offline data processing. We can see that LIFTus makes a significant reduction in preprocessing time compared to D^{3}L and SANTOS, with average speedup ratios of 5.8X and 20X on four datasets, respectively, but takes about 30% more time than that of Starmie (Subsequent generalization experiments will demonstrate that LIFTus trained on a smaller data lake will outperform Starmie). LIFTus builds the index of column embedding using HNSW [25]. It takes 8, 149, 17, and 52 seconds respectively to build the index on the SANTOS small, SANTOS large, TUS small, and TUS large datasets (the average duration of 5 runs is rounded to an integer). During the query stage, even for the largest dataset, LIFTus can return the results within 1 second, while the average query times of D^{3}L and SANTOS are 84.5s and 28.9s respectively. We also find that the value of k has a negligible impact on the query time of LIFTus and Starmie.

The memory consumption of LIFTus mainly stems from two aspects: model training and the storage of column embeddings. Since the information in the table is obtained through sampling and a batch-by-batch training approach is adopted, the memory consumption of model training does not increase with the growth of the data scale (the number of tables, columns, and rows). After obtaining the column embeddings, LIFTus builds HNSW indexes on these embeddings to support efficient table union search. The total number of columns in the data lake influences the memory consumption of this part. Specifically, it takes 49, 965, 116, and 429 MB respectively of memory on the SANTOS small, SANTOS large, TUS small, and TUS large datasets.

To verify the scalability of LIFTus, we randomly select 1M tables with 5M columns from the WDC dataset [58]. We find that the column embedding generation rather than index building takes up the majority of the offline processing time. Specifically, the time of column embedding generation for LIFTus and Stamie increases approximately linear as the data volume increases, which takes 3.7 days and 3.1 days respectively on the selected data set. In the following index construction, LIFTus and Starmie require 1.5 hours and 39 GB of storage space to create index of 5M column embeddings. At the online query stage, the time consumed for a single query of both LIFTus and Starmie is within 1 second. One strategy to further improve the efficiency of LIFTus is to train a model using partial tables while keeping certain query performance, which will be studied in the following.

C. Evaluation of Generalization

We conduct two kinds of experiments to study the generalization of LIFTus, Starmie [14] and SANTOS [7] in two settings. We record the change of $MAP@k$ and the time t used

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

in preprocessing a smaller data lake, and expect that $MAP@k$ can be kept as high as possible, and t can be highly reduced.

Partial Data Training. The data lakes TUS large and SANTOS large contain a relatively large number of tables, which necessitates a significant amount of training time. To reduce the training time and validate model's generalization, we assume that only a portion of the randomly selected tables are available for training. We train the model on partial tables, ranging from 5%, 10%, 25%, 50%, 75%, and 100%. For LIFTus and Starmie [14], only the randomly selected tables and their augmented versions were used when training the column representation learning model. For SANTOS [7], only the randomly selected tables were used when constructing the synthesized knowledge base.

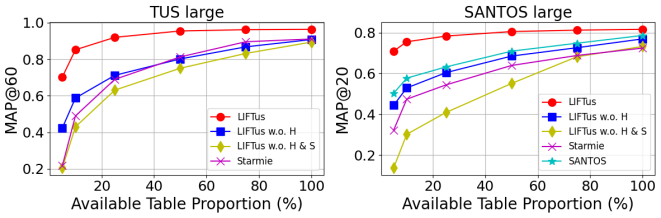


Fig. 6. $MAP@k$ of methods in partial table training.

Fig. 6 reports the $MAP@k$ results in the setting of partial table training, where the red curve corresponding to LIFTus consistently occupies the highest position. We can see that from right to left, $MAP@k$ decreases as the number of tables available for training decreases, but the advantages of LIFTus are most obvious over other methods using the lowest training proportions. In addition, we observe that LIFTus trained with 25% tables can produce better results than other competitors with 100% tables. The results also show that the HCAI aided by aspect-aware statistics plays an important role in the partial data training.

Cross-dataset Transferring. To further validate the generalization of LIFTus, a more challenging experiment is conducted. We hypothesize that only TUS small (denoted as "t") and SANTOS small (denoted as "s") are available for self-supervised training. After the model is learned, we evaluate the model on the unseen and larger data lakes, TUS large (denoted as "T") and SANTOS large (denoted as "S") respectively.

Table V reports the results of cross-dataset transferring setting. "s $\rightarrow$ T" refers to the model self-supervised training in SANTOS small and evaluating in TUS large. As can be seen from the comparison with Table IV, LIFTus is the least affected in $MAP@k$ metric, while other representation learning-based methods demonstrate suboptimal performance. This demonstrates the generalization of LIFTus, enabling it to achieve better performance in larger, unseen data lakes. Additionally, Table V also reports that the advantage of LIFTus diminishes if the HCAI and SDAI components are removed.

Fig. 7 further illustrates the preprocessing time for various methods. A method named with the suffix "-trans" indicates that the method was self-supervised trained on a small data

TABLE V
CROSS-DATASET TRANSFER RESULT

Methods	t $\rightarrow$ T $MAP@60$	s $\rightarrow$ T $MAP@60$	s $\rightarrow$ S $MAP@20$	t $\rightarrow$ S $MAP@20$
Starmie [14]	0.886	0.858	0.532	0.503
LIFTus w.o. H	0.908	0.873	0.630	0.527
LIFTus w.o. H&S	0.870	0.795	0.507	0.457
LIFTus	**0.958**	**0.942**	**0.742**	**0.634**

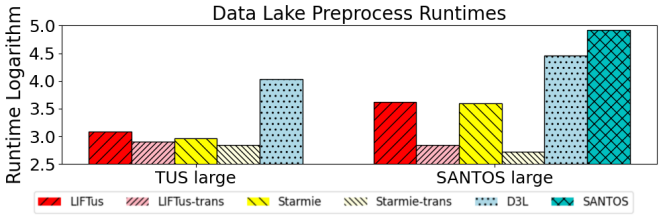


Fig. 7. Preprocessing time for various methods.

lake and applied on a large data lake ("t $\rightarrow$ T" and "s $\rightarrow$ S"). We can see that the preprocessing time for LIFTus is significantly reduced after employing the learned model on the small data, achieving up to a **2X speedup** on the TUS large dataset and a **6X speedup** on the SANTOS large, while there is only a slight decline in $MAP@k$. Fig. 7 also shows the comparison between LIFTus trained on small dataset (LIFTus-trans) and Starmie trained on full dataset. In SANTOS large dataset, LIFTus-trans achieves a **5.6X speedup** while maintaining a **0.018 (2.5%) increase** in $MAP@20$. Similarly, on the TUS large dataset, LIFTus-trans attains a **1.2X speedup**, along with a **0.047 (5.2%) improvement** in $MAP@60$. In all, LIFTus-trans achieves a commendable balance between effectiveness and preprocessing time on both large datasets.

D. Effects of Components in LIFTus

Effects of Pattern Encoder. Table VI shows that the removal of the pattern encoder of LIFTus results in a decrease in $MAP@k$, ranging from 0.7% to 10.7% across the four datasets. Similarly, $R@k$ decreased from 0.6% to 7.3% on the TUS small, TUS large and SANTOS small datasets, respectively. These results underscore the critical role of pattern information in discovering unionable tables. Furthermore, it is evident from these results that the character-level pattern encoder is effective in modeling such an aspect.

TABLE VI
PERFORMANCE COMPARISON OF DIFFERENT VERSIONS OF LIFTUS

Methods	TUS small		TUS large		SANTOS small		SANTOS large
	$MAP@k$	$R@k$	$MAP@k$	$R@k$	$MAP@k$	$R@k$	$MAP@k$
w.o. PE	0.961	0.292	0.907	0.243	0.964	0.704	0.729
w.o. NE	0.971	0.295	0.959	0.257	0.951	0.692	0.757
w.o. H	0.966	0.294	0.908	0.244	0.945	0.687	0.769
w.o. H&S	0.948	0.290	0.894	0.235	0.934	0.680	0.732
LIFTus	**0.976**	**0.298**	**0.964**	**0.262**	**0.971**	**0.708**	**0.816**

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

Effects of Number Encoder. Table VI reports the effects of number encoder. The removal of such an encoder results in decreases in metrics of LIFTus, but these reductions are not compelling. We guess that it is due to the non-numerical columns playing dominant roles in the TUS, and thus the removal of the number encoder does not impact results seriously, in which the non-numerical column means the column with its characters "0"-"9" below 5% of the total characters.

We further design the following experiment to investigate the effects of the number encoder. Specifically, we progressively remove non-numerical columns from TUS large and SANTOS large datasets at proportions of 0%, 25%, 50%, 75%, and 100%, and then evaluate the changes in MAP scores.

Fig. 8 illustrates the variations of $MAP@k$ with respect to the non-numerical column removal ratio for LIFTus, LIFTus w.o. NE, SANTOS, and Starmie on two datasets. We can see the performance gap between LIFTus and LIFTus w.o. NE, which verifies the effects of the number encoder.

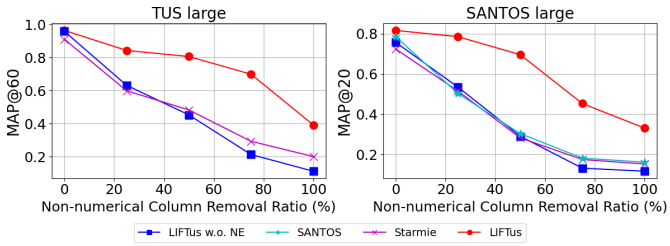


Fig. 8. $MAP@k$ of LIFTus against baselines in different non-numerical column removal ratio.

Effects of SDAI and HCAI. Table VI shows that $MAP@k$ for LIFTus w.o. H&S decreased from 2.9% to 10.3% across four datasets compared to LIFTus, and $R@k$ on the TUS small, TUS large, and SANTOS small datasets decreased by 2.7%, 10.3%, and 4.0%, respectively. We also test the variants where the statistics in SDAI are replaced by random values. The results indicate that both $MAP@k$ and $R@k$ have decreased by at least 8% across all datasets, and the variance of the results obtained from multiple runs is relatively large. Additionally, as illustrated in Fig. 6, $MAP@k$ scores for both LIFTus w.o. H and LIFTus w.o. H&S significantly decline compared to LIFTus, as the proportion of tables available for training decreases. This demonstrates the important roles of both SDAI and HCAI in the effectiveness of LIFTus.

Visualization of HCAI. We explain the execution of HCAI by demonstrating it on a column from the SANTOS small dataset. To demonstrate this, we list some rows of the column, namely "`Adrian 0115`", "`Adrian 029`", "`Holton 1002(G)`", "`Holton 1059(G)`" and "`Phillips Farm 42`". Intuitively, the common prefix patterns and semantically meaningful words are the prominent features of this column.

The aspect extractor first transforms the information in this column into sampled units of 4 aspects. The first layer of HCAI runs the self-attention mechanism within each aspect. After that, all the sampled units in the 4 aspects integrate the information within their respective aspects.

By executing cross-attention with F_0 from SDAI, the weights of all sampled units within 4 aspects are learned in the second layer of HCAI. In Fig. 9, we use a heatmap to show the cross-attention matrix (scaled $F_0 F_{cat}^T$). As illustrated, LIFTus pay more attention to the pattern and word aspects, which is consistent with the intuitive observation of this column.

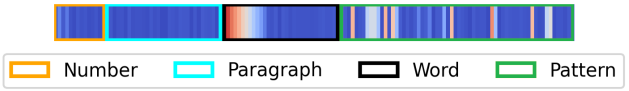


Fig. 9. Cross-attention matrix (scaled $F_0 F_{cat}^T$) in the second layer of HCAI. Weight from high to low, corresponding color from red to blue.

Impact of Different Column Type Distribution. To study the impact of column type on LIFTus, we integrate 4 datasets in Table III to obtain data lake D_{all}. For each data type $t \in \{integer, float, string, longtext\}$, we create a backup D_t of D_{all} with all columns in other type removed. Note that we determine the types of columns through heuristic rules since all tables in D_{all} are csv-formatted. We perform TUS in these 4 data lakes.

TABLE VII
RESULT OF DIFFERENT COLUMN TYPE

Methods	$D_{integer}$ $MAP@10$	D_{float} $MAP@10$	D_{string} $MAP@10$	$D_{longtext}$ $MAP@10$
Starmie [14]	0.356	0.119	0.712	0.821
LIFTus w.o. PE	0.403	0.298	0.694	0.803
LIFTus w.o. NE	0.296	0.134	0.739	0.821
LIFTus w.o. H	0.379	0.203	0.688	0.814
LIFTus w.o. H&S	0.173	0.086	0.542	0.681
LIFTus	**0.448**	**0.373**	**0.766**	**0.837**

The results are shown in Table VII. It can be observed that LIFTus demonstrates the best performance under 4 data lakes with different column types. The gap is more evident in data lakes with integer and float column types. Removing the relevant components of LIFTus leads to a decrease in the results. This indicates that each component we designed for LIFTus is indispensable.

V. CONCLUSION

This paper proposes a method named LIFTus to fully explore different aspects adaptively of individual columns to handle TUS. With the observation that tables in the data lake contain massive non-linguistic data, including the domain-specific strings and numerical data, LIFTus designs a character-level encoder on the extracted wildcard patterns from the domain-specific strings, and devises a number encoder to capture the distribution and magnitude features of numerical data. All encoders are integrated into a representation with the proposed aspect-relevant statistics-aided hierarchical cross-attention mechanism. The experiments illustrate that LIFTus outperforms the competitors in terms of effectiveness and efficiency, and achieves much better generalization capability.

ACKNOWLEDGMENTS

This work is supported by NSFC (No. 62272008), ZTE-PKU joint program (IA20240723020-PO0001), and National Key Laboratory of Data Space Technology and System.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] F. Nargesian, E. Zhu, R. J. Miller, K. Q. Pu, and P. C. Arocena, "Data lake management: challenges and opportunities," *Proceedings of the VLDB Endowment*, vol. 12, no. 12, pp. 1986–1989, 2019.

[2] R. J. Miller, "Open data integration," *Proceedings of the VLDB Endowment*, vol. 11, no. 12, pp. 2130–2139, 2018.

[3] N. Miloslavskaya and A. Tolstoy, "Big data, fast data and data lake concepts," *Procedia Computer Science*, vol. 88, pp. 300–305, 2016.

[4] E. Rahm and P. A. Bernstein, "A survey of approaches to automatic schema matching," *the VLDB Journal*, vol. 10, pp. 334–350, 2001.

[5] R. C. Fernandez, Z. Abedjan, F. Koko, G. Yuan, S. Madden, and M. Stonebraker, "Aurum: A data discovery system," in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1001–1012.

[6] O. Lehmberg and C. Bizer, "Stitching web tables for improving matching quality," *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1502–1513, 2017.

[7] A. Khatiwada, G. Fan, R. Shraga, Z. Chen, W. Gatterbauer, R. J. Miller, and M. Riedewald, "Santos: Relationship-based semantic table union search," *Proceedings of the ACM on Management of Data*, vol. 1, no. 1, pp. 1–25, 2023.

[8] H. Mehmood, E. Gilman, M. Cortes, P. Kostakos, A. Byrne, K. Valta, S. Tekes, and J. Riekki, "Implementing big data lake for heterogeneous data sources," in *2019 ieee 35th international conference on data engineering workshops (icdew)*. IEEE, 2019, pp. 37–44.

[9] F. Nargesian, E. Zhu, K. Q. Pu, and R. J. Miller, "Table union search on open data," *Proceedings of the VLDB Endowment*, vol. 11, no. 7, pp. 813–825, 2018.

[10] A. Bogatu, A. A. Fernandes, N. W. Paton, and N. Konstantinou, "Dataset discovery in data lakes," in *2020 ieee 36th international conference on data engineering (icde)*. IEEE, 2020, pp. 709–720.

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019, pp. 4171–4186.

[12] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever *et al.*, "Improving language understanding by generative pre-training," 2018.

[13] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[14] G. Fan, J. Wang, Y. Li, D. Zhang, and R. J. Miller, "Semantics-aware dataset discovery from data lakes with contextualized column-based representation learning," *Proceedings of the VLDB Endowment*, vol. 16, no. 7, pp. 1726–1739, 2023.

[15] X. Hu, S. Wang, X. Qin, C. Lei, Z. Shen, C. Faloutsos, A. Katsifodimos, G. Karypis, L. Wen, and S. Y. Philip, "Automatic table union search with tabular representation learning," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 3786–3800.

[16] T. Cong, F. Nargesian, and H. Jagadish, "Pylon: Semantic table union search in data lakes," *arXiv preprint arXiv:2301.04901*, 2023.

[17] K. Pal, A. Khatiwada, R. Shraga, and R. J. Miller, "Generative benchmark creation for table union search," *arXiv preprint arXiv:2308.03883*, 2023.

[18] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever *et al.*, "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.

[19] F. M. Suchanek, G. Kasneci, and G. Weikum, "Yago: a core of semantic knowledge," in *Proceedings of the 16th international conference on World Wide Web*, 2007, pp. 697–706.

[20] J. Lehmann, R. Isele, M. Jakob, A. Jentzsch, D. Kontokostas, P. N. Mendes, S. Hellmann, M. Morsey, P. Van Kleef, S. Auer *et al.*, "Dbpedia–a large-scale, multilingual knowledge base extracted from wikipedia," *Semantic web*, vol. 6, no. 2, pp. 167–195, 2015.

[21] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2016, pp. 1715–1725.

[22] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," *Proceedings of the VLDB Endowment*, vol. 14, no. 1, pp. 50–60, 2020.

[23] M. Akhtar, A. Shankarampeta, V. Gupta, A. Patil, O. Cocarascu, and E. Simperl, "Exploring the numerical reasoning capabilities of language models: A comprehensive analysis on tabular data," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 15 391–15 405.

[24] J. Yan, B. Zheng, H. Xu, Y. Zhu, D. Chen, J. Sun, J. Wu, and J. Chen, "Making pre-trained language models great on tabular prediction," in *The Twelfth International Conference on Learning Representations*.

[25] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[26] A. Joulin, É. Grave, P. Bojanowski, and T. Mikolov, "Bag of tricks for efficient text classification," in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, 2017, pp. 427–431.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[28] Z. Miao and J. Wang, "Watchog: A light-weight contrastive learning based framework for column annotation," *Proceedings of the ACM on Management of Data*, vol. 1, no. 4, pp. 1–24, 2023.

[29] M. Hulsebos, K. Hu, M. Bakker, E. Zgraggen, A. Satyanarayan, T. Kraska, Ç. Demiralp, and C. Hidalgo, "Sherlock: A deep learning approach to semantic data type detection," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 1500–1508.

[30] D. Zhang, M. Hulsebos, Y. Suhara, Ç. Demiralp, J. Li, and W.-C. Tan, "Sato: contextual semantic type detection in tables," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 1835–1848, 2020.

[31] S. Mudgal, H. Li, T. Rekatsinas, A. Doan, Y. Park, G. Krishnan, R. Deep, E. Arcaute, and V. Raghavendra, "Deep learning for entity matching: A design space exploration," in *Proceedings of the 2018 international conference on management of data*, 2018, pp. 19–34.

[32] D. Wang, P. Shiralkar, C. Lockard, B. Huang, X. L. Dong, and M. Jiang, "Tcn: Table convolutional network for web table interpretation," in *Proceedings of the Web Conference 2021*, 2021, pp. 4020–4032.

[33] R. Cappuzzo, P. Papotti, and S. Thirumuruganathan, "Creating embeddings of heterogeneous relational datasets for data integration tasks," in *Proceedings of the 2020 ACM SIGMOD international conference on management of data*, 2020, pp. 1335–1349.

[34] D. Yao, Y. Gu, G. Cong, H. Jin, and X. Lv, "Entity resolution with hierarchical graph attention networks," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 429–442.

[35] S. Langenecker, C. Sturm, C. Schalles, and C. Binnig, "Pythagoras: Semantic type detection of numerical data in enterprise data lakes." in *EDBT*, 2024, pp. 725–733.

[36] X. Deng, H. Sun, A. Lees, Y. Wu, and C. Yu, "Turl: Table understanding through representation learning," *ACM SIGMOD Record*, vol. 51, no. 1, pp. 33–40, 2022.

[37] Y. Suhara, J. Li, Y. Li, D. Zhang, Ç. Demiralp, C. Chen, and W.-C. Tan, "Annotating columns with pre-trained language models," in *Proceedings of the 2022 International Conference on Management of Data*, 2022, pp. 1493–1503.

[38] H. Iida, D. Thai, V. Manjunatha, and M. Iyyer, "Tabbie: Pretrained representations of tabular data," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021, pp. 3446–3456.

[39] Z. Wang, H. Dong, R. Jia, J. Li, Z. Fu, S. Han, and D. Zhang, "Tuta: Tree-based transformers for generally structured table pre-training," in *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 1780–1790.

[40] P. Yin, G. Neubig, W.-t. Yih, and S. Riedel, "Tabert: Pretraining for joint understanding of textual and tabular data," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 8413–8426.

[41] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent dirichlet allocation," *Journal of machine Learning research*, vol. 3, no. Jan, pp. 993–1022, 2003.

[42] M. Singh, J. C. Sánchez, S. Gulwani, V. Le, C. Negreanu, M. Raza, and G. Verbruggen, "Cornet: Learning table formatting rules by example," *Proceedings of the VLDB Endowment*, vol. 16, no. 10, pp. 2632–2644, 2023.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.

[43] M. Singh, J. C. Sanchez, S. Gulwani, V. Le, C. Negreanu, and G. Verbruggen, "Cornet: Learning spreadsheet formatting rules by example," *Proceedings of the VLDB Endowment*, vol. 16, no. 12, pp. 4058–4061, 2023.

[44] S. Langenecker, C. Sturm, C. S. Schalles, and C. Binnig, "Steered training data generation for learned semantic type detection," *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–25, 2023.

[45] W. J. Conover, *Practical nonparametric statistics*. john wiley & sons, 1999, vol. 350.

[46] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE transactions on neural networks*, vol. 20, no. 1, pp. 61–80, 2008.

[47] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2022.

[48] Y. Chang, X. Wang, J. Wang, Y. Wu, L. Yang, K. Zhu, H. Chen, X. Yi, C. Wang, Y. Wang *et al.*, "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, vol. 15, no. 3, pp. 1–45, 2024.

[49] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[50] V. Borisov, K. Sessler, T. Leemann, M. Pawelczyk, and G. Kasneci, "Language models are realistic tabular data generators," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=cEygmQNOeI

[51] R. Kohavi and M. Sahami, "Error-based and entropy-based discretization of continuous features." in *KDD*, 1996, pp. 114–119.

[52] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[53] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *International conference on machine learning*. PMLR, 2020, pp. 1597–1607.

[54] C. Jiang, Z. Nian, K. Guo, S. Chu, Y. Zhao, L. Shen, and K. Tu, "Learning numeral embedding," in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 2586–2599.

[55] H. Duan, Y. Yang, and K. Y. Tam, "Learning numeracy: a simple yet effective number embedding approach using knowledge graph," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021, pp. 2597–2602.

[56] X. Wei, T. Zhang, Y. Li, Y. Zhang, and F. Wu, "Multi-modality cross attention network for image and sentence matching," in *CVPR*, 2020.

[57] W.-Y. Lee, L. Jovanov, and W. Philips, "Cross-modality attention and multimodal fusion transformer for pedestrian detection," in *European Conference on Computer Vision*. Springer, 2022, pp. 608–623.

[58] O. Lehmberg, D. Ritze, R. Meusel, and C. Bizer, "A large public corpus of web tables containing time and context metadata," in *Proceedings of the 25th international conference companion on world wide web*, 2016, pp. 75–76.

Authorized licensed use limited to: The University of Utah. Downloaded on January 17,2026 at 03:40:17 UTC from IEEE Xplore. Restrictions apply.